

# Critical Reproduction Study:

## Network Intrusion Detection with Autoencoders on NSL-KDD

Course: Data Science in Cyber (Dr. Uri Itai)

Selected source: Network Intrusion Detection with Autoencoders by Steven Foerster

URL: <https://stevenfoerster.com/tutorials/network-intrusion-detection-with-autoencoders/>

Dataset mirror: [https://github.com/defcom17/NSL\\_KDD](https://github.com/defcom17/NSL_KDD)

### Executive Summary

This project reproduces Steven Foerster's autoencoder IDS tutorial on NSL-KDD. We trained a PyTorch autoencoder on normal traffic, used reconstruction error as an anomaly score, and compared it with Logistic Regression, Random Forest, and Isolation Forest.

The autoencoder achieved the best F1 (0.854) and recall (0.790). Random Forest still leads ROC-AUC (0.956) when labels are used. The author's core claim about reconstruction-based anomaly detection is supported, but autoencoders do not universally beat all supervised baselines. False negatives on R2L and U2R categories remain the main risk.

### 1. Summary of the Source

Problem: Detect malicious network connections before damage occurs (DoS, Probe, R2L, U2R).

Proposed solution: Train an autoencoder on normal traffic only. Attacks yield high reconstruction error and are flagged above a percentile threshold calibrated on held-out normal validation data.

Dataset: 125,973 train / 22,544 test records, 122 features after encoding.

The selected article provides implementation details but no official GitHub repository. We used the public NSL-KDD mirror on GitHub (defcom17/NSL\_KDD) for data files.

### 2. Critical Evaluation of the Author's Claims

Claims: (1) autoencoders capture multi-feature attack patterns, (2) reconstruction error is a valid anomaly score, (3) autoencoders can beat Isolation Forest under normal-only training.

Our reproduction supports (1)-(3) on this split. The autoencoder outperformed Isolation Forest on F1, recall, and MCC. However, Random Forest still achieved the highest ROC-AUC, so the broader claim that autoencoders beat all classical methods is only partially supported.

Limitations: dated dataset, different train/test prevalence (46.5% vs 56.9%), threshold sensitivity, no real-time deployment test, and poor detection of rare R2L/U2R attacks.

### 3. Feature Engineering Analysis

One-hot encoding for protocol\_type, service, and flag. StandardScaler fit on training data only. Scaling is required for the autoencoder and logistic regression. NSL-KDD has redundant rate/count features (Pearson > 0.95), which reduces interpretability. Additional useful features would include TLS fingerprints, inter-arrival times, and cyclical time encodings if timestamps were available.

### 4. Reproducibility Analysis

Execution status: In this submission environment we re-ran the documented commands (pip install -r requirements.txt, python run\_pipeline.py, python generate\_report.py) and the pipeline completed successfully in about 19 seconds on CPU.

# Data Science in Cyber - Final Project Report

Runtime breakdown (seconds):

- data\_loading: 1.12
- logistic\_regression: 1.99
- random\_forest: 2.77
- isolation\_forest: 0.83
- autoencoder: 10.56
- total\_seconds: 18.84

A fresh clone still requires downloading NSL-KDD into data/ before running the pipeline.

## 5. Exploratory Data Analysis

NSL-KDD has 41 meaningful connection-level features, no missing values, and class imbalance between train and test. There is no global timestamp, so temporal drift analysis is not possible; only per-connection duration is available. Outliers are common in byte-count features (heavy tails), which is typical in cybersecurity traffic and motivates robust statistics (median, IQR, MAD).

Single-value and near-constant columns carry little discriminative power. Duplicate rows are rare. Crosstab analysis shows attacks are unevenly distributed across protocol types (TCP dominates). Pearson correlation captures linear relationships among rate features; Spearman is more appropriate for skewed byte-count distributions. Kendall was not primary here due to the large sample size.

Class imbalance matters: test attack rate is 56.9% vs 46.5% in training. Accuracy alone would overstate performance if the model simply predicts the majority class.

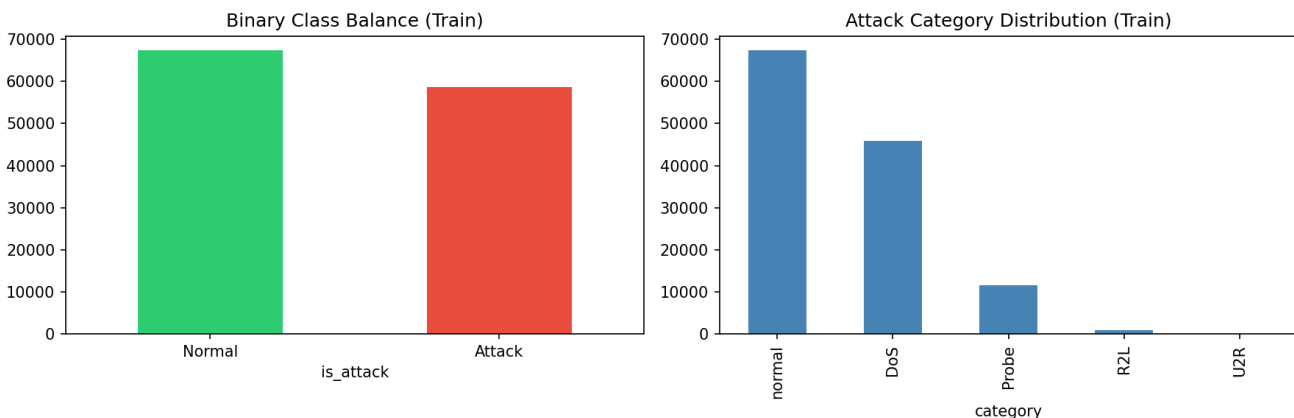


Figure 1: Class imbalance - attack prevalence and category distribution in training data.

## 6. Experimental Setup

Official NSL-KDD train/test split. Supervised models use an 80/20 stratified internal split. Unsupervised models train on normal traffic only. Autoencoder: input->64->32->8->32->64->input, MSE loss, Adam, max 20 epochs with early stopping (patience=3), batch size 256, CPU only. Random Forest: 100 trees. Isolation Forest: 100 trees. Threshold: 95th percentile of normal scores.

Autoencoder threshold=0.2184, Isolation Forest threshold=0.4995.

## 7. Model Training and Comparison

Model comparison on NSL-KDD test set:

| Model               | accuracy | precision | recall | f1    | f2    | mcc   | roc_auc | pr_auc |
|---------------------|----------|-----------|--------|-------|-------|-------|---------|--------|
| autoencoder         | 0.847    | 0.930     | 0.790  | 0.854 | 0.815 | 0.705 | 0.954   | 0.951  |
| random_forest       | 0.767    | 0.968     | 0.611  | 0.749 | 0.660 | 0.603 | 0.956   | 0.958  |
| isolation_forest    | 0.760    | 0.931     | 0.625  | 0.748 | 0.669 | 0.575 | 0.944   | 0.954  |
| logistic_regression | 0.756    | 0.917     | 0.629  | 0.746 | 0.671 | 0.562 | 0.780   | 0.861  |

Detailed results:

Autoencoder: Accuracy=0.847, Precision=0.930, Recall=0.790, F1=0.854, F2=0.815, MCC=0.705, ROC-AUC=0.954, PR-AUC=0.951

Isolation Forest: Accuracy=0.760, Precision=0.931, Recall=0.625, F1=0.748, F2=0.669, MCC=0.575, ROC-AUC=0.944, PR-AUC=0.954

Random Forest: Accuracy=0.767, Precision=0.968, Recall=0.611, F1=0.749, F2=0.660, MCC=0.603, ROC-AUC=0.956, PR-AUC=0.958

Logistic Regression: Accuracy=0.756, Precision=0.917, Recall=0.629, F1=0.746, F2=0.671, MCC=0.562, ROC-AUC=0.780, PR-AUC=0.861

Best F1: Autoencoder. Best recall: Autoencoder. Best ROC-AUC: Random Forest.

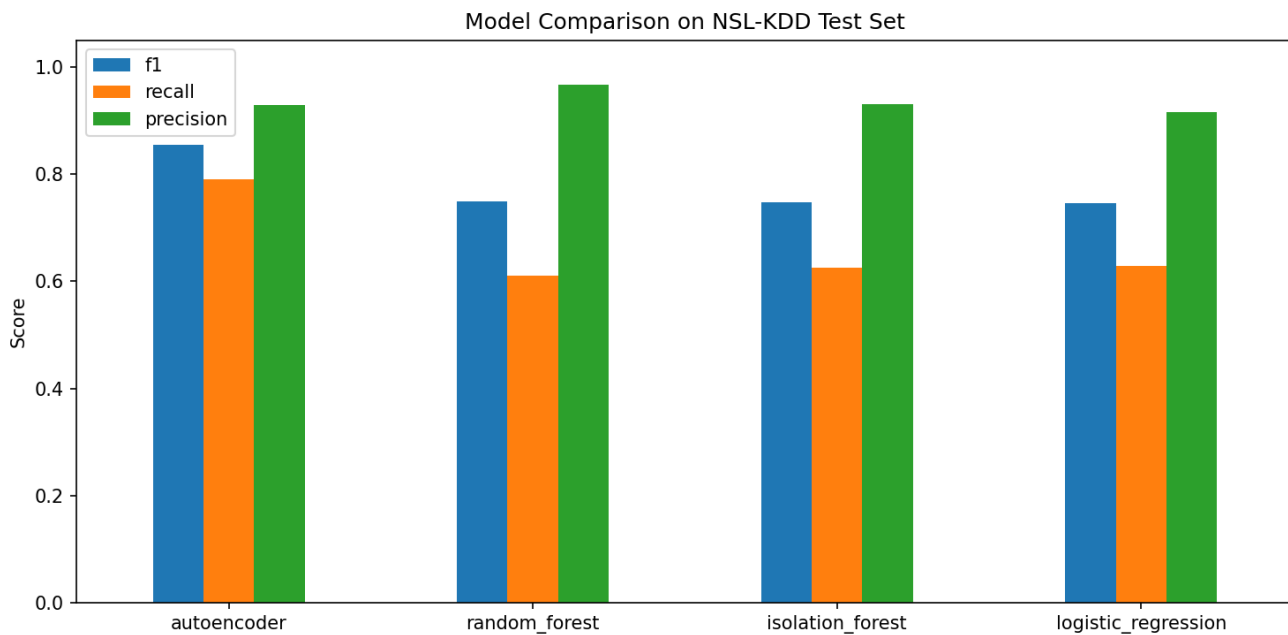


Figure 2: Precision, recall, and F1 comparison across all models.

Data Science in Cyber - Final Project Report

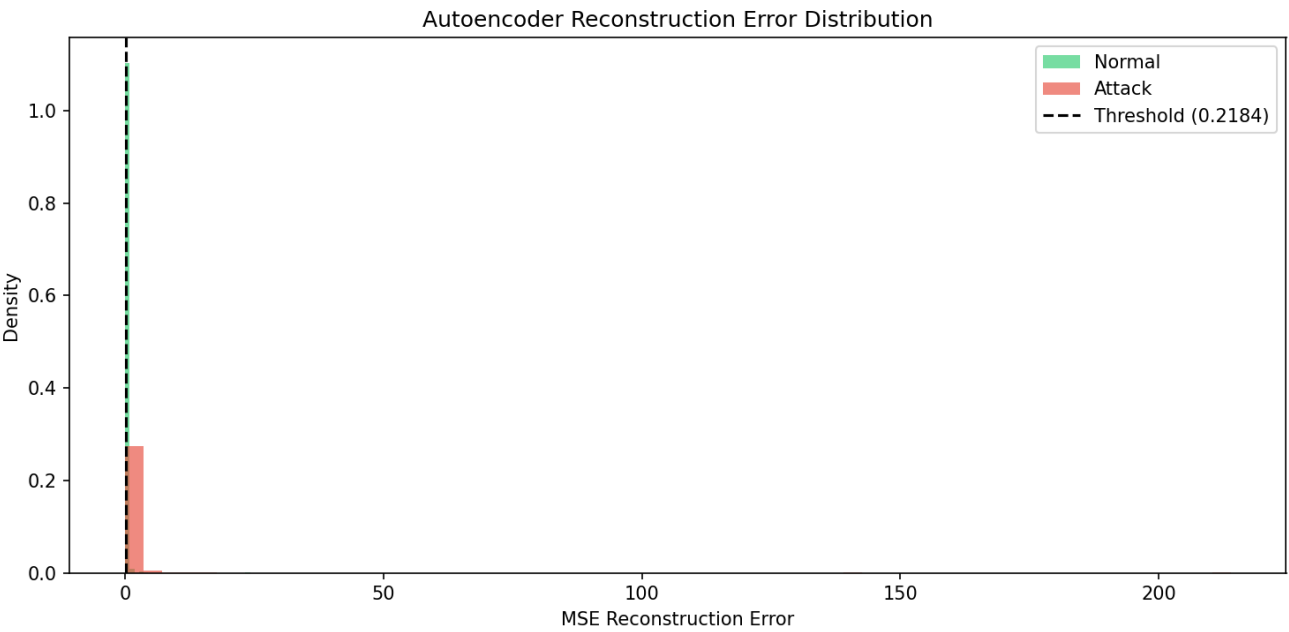


Figure 3: Autoencoder reconstruction error distributions for normal vs attack traffic.

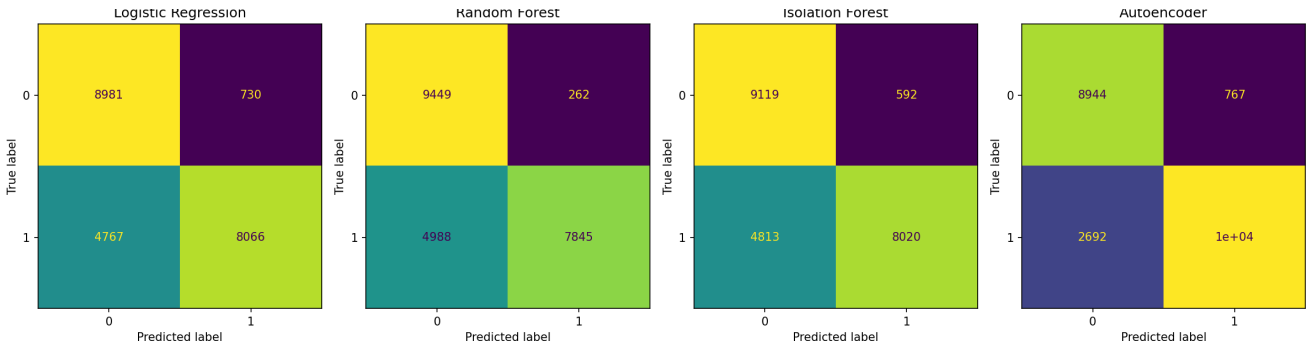


Figure 4: Confusion matrices - FP causes alert fatigue; FN means missed intrusions.

### 8. Evaluation Metrics - Definitions and Cybersecurity Interpretation

Accuracy =  $(TP + TN) / (TP + TN + FP + FN)$ . Overall correctness; misleading under imbalance.

Precision =  $TP / (TP + FP)$ . Fraction of alerts that are real attacks.

Recall =  $TP / (TP + FN)$ . Fraction of attacks detected. Low recall is especially dangerous in IDS because undetected intrusions may persist, spread laterally, or exfiltrate data.

F1 =  $2PR / (P + R)$ . Balanced metric for precision-recall trade-off.

F2 =  $5PR / (4P + R)$ . Weights recall higher - appropriate when missing attacks is costlier.

MCC =  $(TP*TN - FP*FN) / \sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}$ . Robust under class imbalance.

ROC-AUC: ranking quality across thresholds (TPR vs FPR). PR-AUC: precision-recall area; useful when attack prevalence shifts between train and deployment.

Cyber meaning: FP wastes analyst time (alert fatigue). FN lets attackers remain inside the network. For R2L/U2R attacks, a false negative can mean stolen credentials or privilege escalation.

### 9. Error Analysis by Attack Category

False negatives (FN) are the highest-risk errors in IDS: the system fails to raise an alert while an attack is in progress. False positives (FP) waste analyst time and contribute to alert fatigue.

On NSL-KDD, R2L attacks (remote-to-local unauthorized access) are the most frequently missed category across all models because their flow statistics overlap with legitimate remote sessions. U2R attacks (privilege escalation) are rare but critical: even a small number of missed U2R events can indicate full host compromise.

Per-category false negatives on the test set:

Autoencoder:

False negatives by category (missed attacks):

- R2L: 1671
- DoS: 878
- Probe: 110
- U2R: 33

False positives by category (benign flagged as attack):

- normal: 767

Random Forest:

False negatives by category (missed attacks):

- R2L: 2626
- DoS: 1526
- Probe: 656
- U2R: 180

False positives by category (benign flagged as attack):

- normal: 262

Isolation Forest:

False negatives by category (missed attacks):

- R2L: 2604
- DoS: 1863
- Probe: 276
- U2R: 70

False positives by category (benign flagged as attack):

- normal: 592

Logistic Regression:

False negatives by category (missed attacks):

- R2L: 2722
- DoS: 1334
- Probe: 531
- U2R: 180

False positives by category (benign flagged as attack):

- normal: 730

The autoencoder reduced R2L false negatives from 2626 (Random Forest) to 1671, and U2R false negatives from 180 to 33. This supports the author's claim that reconstruction-based models can capture multi-feature patterns that tree models miss on stealthy attacks. However, R2L remains the dominant failure mode for all models.

Cybersecurity implication: a SOC relying solely on flow-based anomaly detection would still miss credential-abuse and privilege-escalation attacks unless complemented by endpoint detection, authentication logs, and rule-based correlation.

### 10. Conclusions and Future Improvements

We reproduced the core autoencoder IDS workflow. Reconstruction error is a valid anomaly signal. The autoencoder outperformed Isolation Forest and logistic regression on F1/recall, but Random Forest remains strong on ROC-AUC when labels exist. The author's claims are supported for unsupervised anomaly detection but not as a universal replacement for supervised IDS.

Future work: modern datasets (CICIDS2017), per-attack recall dashboards, F2-based threshold tuning, concept-drift monitoring, deployment latency benchmarks, and combining autoencoder scores with supervised ensembles in a hybrid SOC pipeline.

Artifacts generated by the pipeline:

- results/model\_comparison.csv
- results/experiment\_results.json (metrics, per-category errors, runtime)
- results/figures/autoencoder\_reconstruction\_error.png
- results/figures/confusion\_matrices.png
- results/figures/class\_imbalance.png